BÁO CÁO BÀI 2 PHẦN CÁ NHÂN

Trao đổi dữ liệu MQTT bằng Java

Sinh viên: Nguyễn Trí Hiếu

MSSV: 202416203

1 Nội dung thực hiện

Chương trình Java gửi dữ liệu cảm biến mô phỏng lên broker và nhận bản tin MQTT để hiển thị đủ các trường JSON. Báo cáo trình bày ba tệp mã nguồn chính, giải thích các khối xử lý và minh chứng thực hành về gửi nhận dữ liệu, xử lý JSON lỗi, Last Will, retained và tự kết nối lại.

Thông số	Giá trị sử dụng
Môi trường	JDK 17 trở lên; Eclipse Paho 1.2.5; JSON-java 20240303
Broker mặc định	tcp://broker.hivemq.com:1883
Topic gửi dữ liệu	it4735e/202416203/telemetry
Topic đăng ký nhận	it4735e/202416203/#
Topic trạng thái	it4735e/202416203/status
Gửi dữ liệu	QoS 1; packet_no bắt đầu từ 126; thời gian chờ mặc định 5 giây giữa các lượt gửi

Vai trò của ba tệp

ClientConfig.java kiểm tra tham số và tạo topic theo MSSV. Telemetry.java tạo dữ liệu mô phỏng, chuyển đổi JSON và định dạng kết quả. MqttAssignment.java điều khiển kết nối, gửi nhận, phục hồi kết nối và dừng chương trình.

Luồng hoạt động

Đọc tham số → kết nối broker → đăng ký topic → gửi trạng thái online → gửi telemetry định kỳ. Mỗi bản tin nhận được sẽ được phân loại thành trạng thái hoặc dữ liệu JSON để hiển thị.

Dữ liệu gồm 7 trường: id, name, packet_no, temperature, humidity, tds và pH. Với QoS 1, bản tin nhận có thể trùng; packet_no được đặt lại về 126 mỗi khi chạy chương trình mới.

2 Cấu hình kết nối

Tệp ClientConfig.java

Khai báo và kiểm tra cấu hình

Dòng 1 đến 24. Record lưu MSSV, broker và khoảng chờ gửi. Hàm khởi tạo yêu cầu MSSV hợp lệ, khoảng chờ dương và địa chỉ broker đúng dạng tcp://host:port; cấu hình sai sẽ phát sinh lỗi để báo cho người dùng.

```
package vn.edu.iot;

import java.net.URI;

/** Command-line options; MSSV is required so no one accidentally uses a shared topic. */
public record ClientConfig(String studentId, String broker, int intervalSeconds) {
    public static final String DEFAULT_BROKER = "tcp://broker.hivemq.com:1883";

    public ClientConfig {
        if (studentId == null || !studentId.matches("[A-Za-z0-9_-]+")) {
            throw new IllegalArgumentException("MSSV must contain only letters, digits, '_' or '-
'.");
        }
        if (intervalSeconds < 1) {
            throw new IllegalArgumentException("Interval must be a positive number of seconds.");
        }
        URI uri = URI.create(broker);
        if (!"tcp".equals(uri.getScheme()) || uri.getHost() == null
                || uri.getPort() < 1 || uri.getPort() > 65535
                || uri.getUserInfo() != null || uri.getQuery() != null || uri.getFragment() != null
                || (uri.getPath() != null && !uri.getPath().isEmpty())) {
            throw new IllegalArgumentException("Broker must be tcp://host:port (for example, "
                    + DEFAULT_BROKER + ").");
        }
    }
```

Đọc tham số dòng lệnh

Dòng 25 đến 53. parse đọc --mssv, --broker và --interval. Broker mặc định là HiveMQ, khoảng chờ là 5 giây. Chương trình báo lỗi nếu gặp tùy chọn lạ, thiếu giá trị hoặc khoảng chờ không phải số nguyên; MSSV phải được cung cấp.

```
    public static ClientConfig parse(String[] args) {
        String studentId = null;
        String broker = DEFAULT_BROKER;
        int interval = 5;
        for (int i = 0; i < args.length; i++) {
            String option = args[i];
            if (!option.equals("--mssv") && !option.equals("--broker") && !option.equals("--
interval")) {
                throw new IllegalArgumentException("Unknown option: " + option);
            }
            if (i + 1 == args.length || args[i + 1].startsWith("--")) {
                throw new IllegalArgumentException("Missing value for " + option);
            }
            String value = args[++i];
            switch (option) {
                case "--mssv" -> studentId = value;
                case "--broker" -> broker = value;
                case "--interval" -> {
                    try {
                        interval = Integer.parseInt(value);
                    } catch (NumberFormatException e) {
                        throw new IllegalArgumentException("Interval must be a positive integer.");
                    }
                }
                default -> throw new IllegalArgumentException("Unknown option: " + option);
            }
```

```
        }
        return new ClientConfig(studentId, broker, interval);
    }
```

Tạo topic riêng theo MSSV

Dòng 54 đến 66. Ba hàm ghép MSSV vào topic gửi telemetry, topic đăng ký có ký tự # và topic trạng thái. Khi dùng MSSV 202416203, các topic trùng với bảng thông số ở phần đầu.

```
    public String telemetryTopic() {
        return "it4735e/" + studentId + "/telemetry";
    }
    public String subscriptionTopic() {
        return "it4735e/" + studentId + "/#";
    }
    public String statusTopic() {
        return "it4735e/" + studentId + "/status";
    }
}
```

3 Dữ liệu cảm biến

Tệp Telemetry.java

Khai báo bản tin và tạo dữ liệu mô phỏng

Dòng 1 đến 18. Record chứa đủ 7 trường. sample tạo nhiệt độ và pH làm tròn một chữ số thập phân, độ ẩm và TDS ngẫu nhiên; packetNo lấy từ bộ đếm gửi. Đây là dữ liệu mô phỏng, chưa đọc từ cảm biến thật.

```
package vn.edu.iot;

import java.util.Locale;
import java.util.concurrent.ThreadLocalRandom;
import org.json.JSONException;
import org.json.JSONObject;
import org.json.JSONTokener;

/** The seven fields required by Part 2. Sensor readings are simulated. */
public record Telemetry(long id, String name, long packetNo, double temperature,
                        double humidity, double tds, double pH) {
    public static Telemetry sample(long packetNo) {
        ThreadLocalRandom random = ThreadLocalRandom.current();
        return new Telemetry(11, "my-java-mqtt-client", packetNo,
                Math.round(random.nextDouble(20, 35) * 10) / 10.0,
                random.nextInt(40, 76), random.nextInt(800, 1201),
                Math.round(random.nextDouble(5, 7) * 10) / 10.0);
    }
```

Chuyển dữ liệu thành JSON

Dòng 19 đến 30. toJson dùng JSONObject để tạo chuỗi gửi đi. Tên khóa packet_no dùng dấu gạch dưới và pH giữ đúng chữ hoa chữ thường theo yêu cầu.

```
    public String toJson() {
        return new JSONObject()
                .put("id", id)
                .put("name", name)
                .put("packet_no", packetNo)
                .put("temperature", temperature)
                .put("humidity", humidity)
                .put("tds", tds)
                .put("pH", pH)
                .toString();
    }
```

Đọc và kiểm tra JSON nhận được

Dòng 31 đến 42. fromJson đọc đối tượng JSON, kiểm tra phần nội dung thừa phía sau rồi lấy đủ các trường. Các hàm get báo lỗi khi thiếu trường hoặc giá trị không thể đọc theo kiểu yêu cầu; lỗi sẽ được xử lý ở lớp MQTT.

```
    public static Telemetry fromJson(String payload) {
        JSONTokener tokens = new JSONTokener(payload);
        JSONObject data = new JSONObject(tokens);
        if (tokens.nextClean() != 0) {
            throw new JSONException("Unexpected content after JSON object.");
        }
        // Required getters also reject missing fields and incompatible types.
        return new Telemetry(data.getLong("id"), data.getString("name"),
                data.getLong("packet_no"), data.getDouble("temperature"),
                data.getDouble("humidity"), data.getDouble("tds"), data.getDouble("pH"));
    }
```

Định dạng kết quả để hiển thị

Dòng 43 đến 50. display tạo hai dòng kết quả chứa đủ thông tin bản tin. Locale.ROOT giữ cách định dạng số nhất quán, các giá trị cảm biến được in với một chữ số thập phân.

```
    public String display() {
        return String.format(Locale.ROOT,
                "id=%d, name=%s, packet_no=%d%n"
                        + "   temperature=%.1f, humidity=%.1f, tds=%.1f, pH=%.1f",
                id, name, packetNo, temperature, humidity, tds, pH);
    }
}
```

4 Chương trình gửi nhận MQTT

Tệp MqttAssignment.java

Thư viện và trạng thái chương trình

Dòng 1 đến 32. Các thư viện hỗ trợ MQTT, JSON, UTF-8 và xử lý đồng thời. QoS đặt bằng 1, thời gian chờ xác nhận tối đa 15 giây. worker thực hiện công việc theo lịch trên một luồng; ready và closing kiểm soát việc gửi, stopped giữ chương trình chờ, bộ đếm bắt đầu từ 126.

```
package vn.edu.iot;

import java.nio.charset.StandardCharsets;
import java.util.Arrays;
import java.util.UUID;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.Executors;
import java.util.concurrent.RejectedExecutionException;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicBoolean;
import org.eclipse.paho.client.mqttv3.IMqttDeliveryToken;
import org.eclipse.paho.client.mqttv3.MqttAsyncClient;
import org.eclipse.paho.client.mqttv3.MqttCallbackExtended;
import org.eclipse.paho.client.mqttv3.MqttConnectOptions;
import org.eclipse.paho.client.mqttv3.MqttException;
import org.eclipse.paho.client.mqttv3.MqttMessage;
import org.eclipse.paho.client.mqttv3.persist.MemoryPersistence;
import org.json.JSONException;

/** One program performs both publishing and subscribing, including both optional extensions. */
public final class MqttAssignment implements AutoCloseable {
    private static final int QOS = 1;
    private static final long MQTT_TIMEOUT_MS = 15_000;
    private final ClientConfig config;
    private final MqttAsyncClient client;
    private final ScheduledExecutorService worker = Executors.newSingleThreadScheduledExecutor();
    private final AtomicBoolean closing = new AtomicBoolean();
    private final CountDownLatch stopped = new CountDownLatch(1);
```

```
    private volatile boolean ready;
    // Only the single worker thread changes this counter. It resets on a new program run.
    private long nextPacketNo = 126;
```

Tạo client và xử lý kết nối thành công

Dòng 33 đến 51. UUID tạo client ID mới cho mỗi lần chạy. Callback connectComplete gọi prepareSession trên worker sau cả kết nối đầu tiên và kết nối lại. Việc chờ xác nhận đăng ký hoặc gửi được chuyển sang worker để không chặn luồng callback của Paho.

```
    MqttAssignment(ClientConfig config) throws MqttException {
        this.config = config;
        String clientId = "java-" + UUID.randomUUID();
        client = new MqttAsyncClient(config.broker(), clientId, new MemoryPersistence());
        client.setCallback(new MqttCallbackExtended() {
            @Override
            public void connectComplete(boolean reconnect, String serverURI) {
                ready = false;
                System.out.println((reconnect ? "Reconnected to " : "Connected to ") + serverURI);
                // SUBACK/PUBACK must be awaited off the Paho callback thread.
                try {
                    worker.execute(MqttAssignment.this::prepareSession);
                } catch (RejectedExecutionException e) {
                    if (!closing.get()) {
                        System.err.println("Cannot prepare session: " + e.getMessage());
                    }
                }
            }
```

Xử lý mất kết nối và nhận bản tin

Dòng 52 đến 74. connectionLost đặt ready về false để tạm ngừng gửi. messageArrived giải mã payload UTF-8 rồi gọi formatReceived. deliveryComplete không cần xử lý thêm vì hàm gửi tự chờ xác nhận trên worker; cuối hàm khởi tạo in thông tin client và topic.

```
            @Override
            public void connectionLost(Throwable cause) {
                ready = false;
                System.err.println("Connection lost: " + (cause == null ? "unknown reason" :
cause.getMessage())
                        + ". Automatic reconnect is enabled; publishing is paused.");
            }
            @Override
            public void messageArrived(String topic, MqttMessage message) {
                String payload = new String(message.getPayload(), StandardCharsets.UTF_8);
                System.out.println(formatReceived(config.statusTopic(), topic, payload));
            }
            @Override
            public void deliveryComplete(IMqttDeliveryToken token) {
                // Publishing below waits for QoS 1 delivery confirmation on the worker thread.
            }
        });
        System.out.println("Client ID: " + clientId);
        System.out.println("Telemetry topic: " + config.telemetryTopic());
        System.out.println("Press Ctrl+C to stop.");
    }
```

Thiết lập MQTT và Last Will

Dòng 75 đến 86. Dùng MQTT 3.1.1, phiên sạch, thời gian kết nối 10 giây và keep alive 15 giây. Tự kết nối lại được bật. Last Will là chuỗi offline trên topic trạng thái, QoS 1 và retained để broker lưu trạng thái gần nhất khi kết nối mất bất thường.

```
    private MqttConnectOptions connectionOptions() {
        MqttConnectOptions options = new MqttConnectOptions();
        options.setMqttVersion(MqttConnectOptions.MQTT_VERSION_3_1_1);
        options.setCleanSession(true);
        options.setConnectionTimeout(10);
```

```
        options.setKeepAliveInterval(15);
        options.setAutomaticReconnect(true);
        // The broker sends this retained message if the connection ends unexpectedly.
        options.setWill(config.statusTopic(), "offline".getBytes(StandardCharsets.UTF_8), QOS,
true);
        return options;
    }
```

Kết nối lần đầu và lên lịch gửi

Dòng 87 đến 106. Nếu lần kết nối đầu thất bại, chương trình chờ 5 giây rồi thử lại. Khi kết nối thành công, scheduleWithFixedDelay lên lịch gửi: sau khi một lượt xử lý kết thúc, chờ thêm khoảng thời gian đã cấu hình rồi chạy lượt tiếp theo. stopped.await giữ chương trình hoạt động đến lúc đóng.

```
    void run() throws InterruptedException {
        // Paho's automatic reconnect handles losses AFTER a successful connection.
        // Retry the first connection ourselves as well.
        while (!closing.get() && !client.isConnected()) {
            try {
                client.connect(connectionOptions()).waitForCompletion(MQTT_TIMEOUT_MS);
            } catch (MqttException e) {
                if (closing.get()) {
                    return;
                }
                System.err.println("Initial connection failed: " + e.getMessage() + ". Retrying in 5
seconds.");
                Thread.sleep(5_000);
            }
        }
        if (!closing.get()) {
            worker.scheduleWithFixedDelay(this::publishTelemetry, 0, config.intervalSeconds(),
TimeUnit.SECONDS);
            stopped.await();
        }
    }
```

Chuẩn bị phiên sau mỗi lần kết nối

Dòng 107 đến 131. prepareSession đăng ký topic có # với QoS 1 và chờ broker xác nhận. Sau đó gửi online với retained rồi bật ready. Vì cleanSession bằng true nên phải đăng ký lại sau mỗi lần kết nối; nếu chuẩn bị phiên lỗi, chương trình thử lại sau 2 giây.

```
    private void prepareSession() {
        if (closing.get() || !client.isConnected()) {
            return;
        }
        try {
            // Clean sessions lose subscriptions; repeat this on EVERY connectComplete callback.
            var subscription = client.subscribe(config.subscriptionTopic(), QOS);
            subscription.waitForCompletion(MQTT_TIMEOUT_MS);
            if (subscription.getGrantedQos()[0] == 128) {
                throw new MqttException(MqttException.REASON_CODE_SUBSCRIBE_FAILED);
            }
            System.out.println("Subscribed: " + config.subscriptionTopic() + " (QoS 1)");
            client.publish(config.statusTopic(), "online".getBytes(StandardCharsets.UTF_8), QOS,
true)
                    .waitForCompletion(MQTT_TIMEOUT_MS);
            System.out.println(">> Published retained online to " + config.statusTopic());
            ready = client.isConnected() && !closing.get();
        } catch (MqttException e) {
            ready = false;
            System.err.println("Session setup failed: " + e.getMessage());
            if (!closing.get()) {
                worker.schedule(this::prepareSession, 2, TimeUnit.SECONDS);
            }
        }
    }
```

Gửi dữ liệu theo lịch và tăng bộ đếm

Dòng 132 đến 149. Chỉ gửi khi client đã sẵn sàng và còn kết nối. Dữ liệu được tạo bằng Telemetry.sample, đổi sang JSON rồi gửi QoS 1, không retained. Bộ đếm tăng ngay khi lệnh publish được chấp nhận, trước khi chờ xác nhận; lỗi xác nhận được in ra và chương trình vẫn tiếp tục.

```
    private void publishTelemetry() {
        if (closing.get() || !ready || !client.isConnected()) {
            return;
        }
        Telemetry reading = Telemetry.sample(nextPacketNo);
        try {
            var delivery = client.publish(config.telemetryTopic(),
                    reading.toJson().getBytes(StandardCharsets.UTF_8), QOS, false);
            // Count a new accepted publish once, even if confirmation is delayed or duplicated.
            nextPacketNo++;
            delivery.waitForCompletion(MQTT_TIMEOUT_MS);
            System.out.println(">> Published packet_no=" + reading.packetNo()
                    + " to " + config.telemetryTopic() + " (QoS 1 confirmed)");
        } catch (MqttException e) {
            System.err.println("Publish not confirmed: " + e.getMessage() + ". Program continues.");
        }
    }
```

Phân loại trạng thái và xử lý JSON lỗi

Dòng 150 đến 161. Bản tin trên topic status được in như văn bản. Các topic còn lại được đọc thành Telemetry và hiển thị đủ trường. JSONException được bắt tại đây để bản tin sai không làm lỗi lan ra callback; bản tin hợp lệ tiếp theo vẫn được xử lý.

```
    /** Catch bad messages here so a JSON error never escapes into Paho's network callbacks. */
    static String formatReceived(String statusTopic, String topic, String payload) {
        if (topic.equals(statusTopic)) {
            return "<< Status on " + topic + ": " + payload;
        }
        try {
            return "<< Received on " + topic + System.lineSeparator() + "   " +
Telemetry.fromJson(payload).display();
        } catch (JSONException e) {
            return "JSON error on " + topic + ": " + e.getMessage() + " (continuing)";
        }
    }
```

Đóng kết nối và giải phóng tài nguyên

Dòng 162 đến 193. compareAndSet bảo đảm việc đóng chỉ thực hiện một lần. Chương trình dừng worker, chủ động gửi offline retained khi còn kết nối rồi ngắt MQTT. Khối finally dọn kết nối, đóng client và mở chốt stopped. Khi thoát bình thường, offline được chương trình tự gửi vì Last Will dành cho mất kết nối bất thường.

```
    @Override
    public void close() {
        if (!closing.compareAndSet(false, true)) {
            return;
        }
        ready = false;
        worker.shutdownNow();
        try {
            if (client.isConnected()) {
                // A normal MQTT disconnect does not trigger Last Will; announce offline ourselves.
                client.publish(config.statusTopic(), "offline".getBytes(StandardCharsets.UTF_8),
QOS, true)
                        .waitForCompletion(3_000);
                client.disconnect(1_000).waitForCompletion(3_000);
            }
        } catch (MqttException e) {
            System.err.println("Shutdown: " + e.getMessage());
        } finally {
```

```
            try {
                // Reset transport state even if a graceful disconnect failed.
                client.disconnectForcibly(0, 1_000);
            } catch (MqttException e) {
                System.err.println("Disconnect cleanup: " + e.getMessage());
            }
            try {
                client.close(true);
            } catch (MqttException e) {
                System.err.println("Close cleanup: " + e.getMessage());
            }
            stopped.countDown();
        }
    }
```

Điểm bắt đầu và xử lý khi thoát

Dòng 194 đến 228. main hỗ trợ --help, đọc cấu hình và báo lỗi tham số. Shutdown hook gọi close khi JVM kết thúc, ví dụ Ctrl+C; try-with-resources và finally bảo đảm dọn tài nguyên. Nếu luồng bị ngắt, chương trình đặt lại cờ ngắt; lỗi tạo client được báo riêng.

```
    public static void main(String[] args) {
        if (Arrays.asList(args).contains("--help")) {
            usage();
            return;
        }
        ClientConfig config;
        try {
            config = ClientConfig.parse(args);
        } catch (IllegalArgumentException e) {
            System.err.println(e.getMessage());
            usage();
            System.exit(2);
            return;
        }
        try (MqttAssignment app = new MqttAssignment(config)) {
            Thread shutdownHook = new Thread(app::close, "mqtt-shutdown");
            Runtime.getRuntime().addShutdownHook(shutdownHook);
            try {
                app.run();
            } finally {
                app.close();
                try {
                    Runtime.getRuntime().removeShutdownHook(shutdownHook);
                } catch (IllegalStateException ignored) {
                    // The JVM is already running the shutdown hook.
                }
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        } catch (MqttException e) {
            System.err.println("Cannot create MQTT client: " + e.getMessage());
            System.exit(1);
        }
    }
```

Hiển thị cách dùng

Dòng 229 đến 234. usage in mẫu lệnh gồm MSSV bắt buộc, khoảng chờ gửi và địa chỉ broker tùy chọn. Khi chạy bài này, thay YOUR_MSSV bằng 202416203.

```
    private static void usage() {
        System.out.println("Usage: java -jar mqtt-assignment-1.0.0.jar --mssv YOUR_MSSV"
                + " [--interval 5] [--broker tcp://broker.hivemq.com:1883]");
    }
}
```

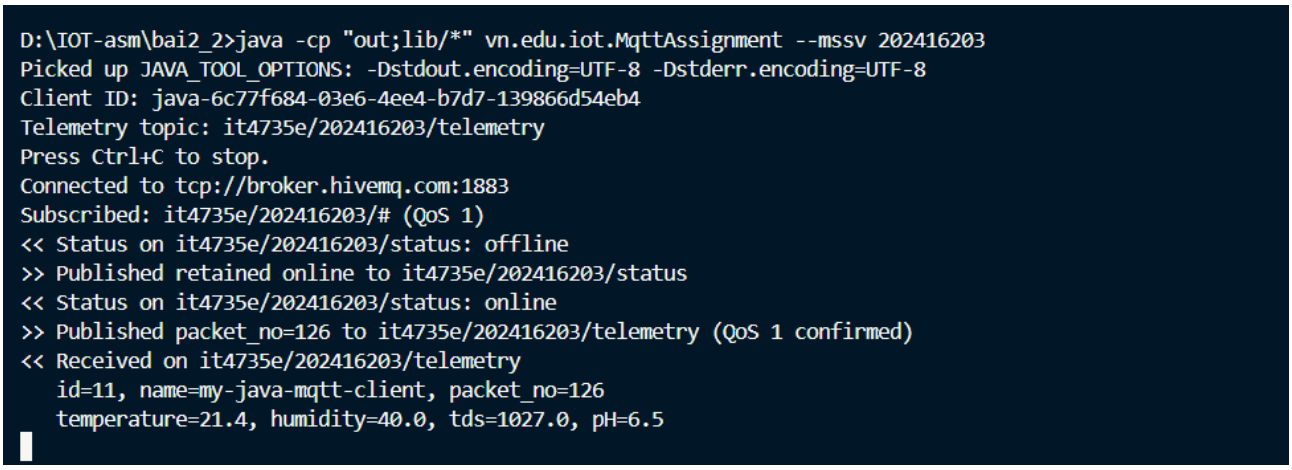
5 Kết quả thực hành và minh chứng

Chương trình sử dụng MSSV 202416203 và broker tcp://broker.hivemq.com:1883. Java đăng ký it4735e/202416203/#; MQTTX được dùng để theo dõi các topic telemetry và status. Lệnh chạy với các thư viện đã có trong thư mục lib:

```
cd D:\IOT-asm\bai2_2
java -cp "out;lib/*" vn.edu.iot.MqttAssignment --mssv 202416203
```

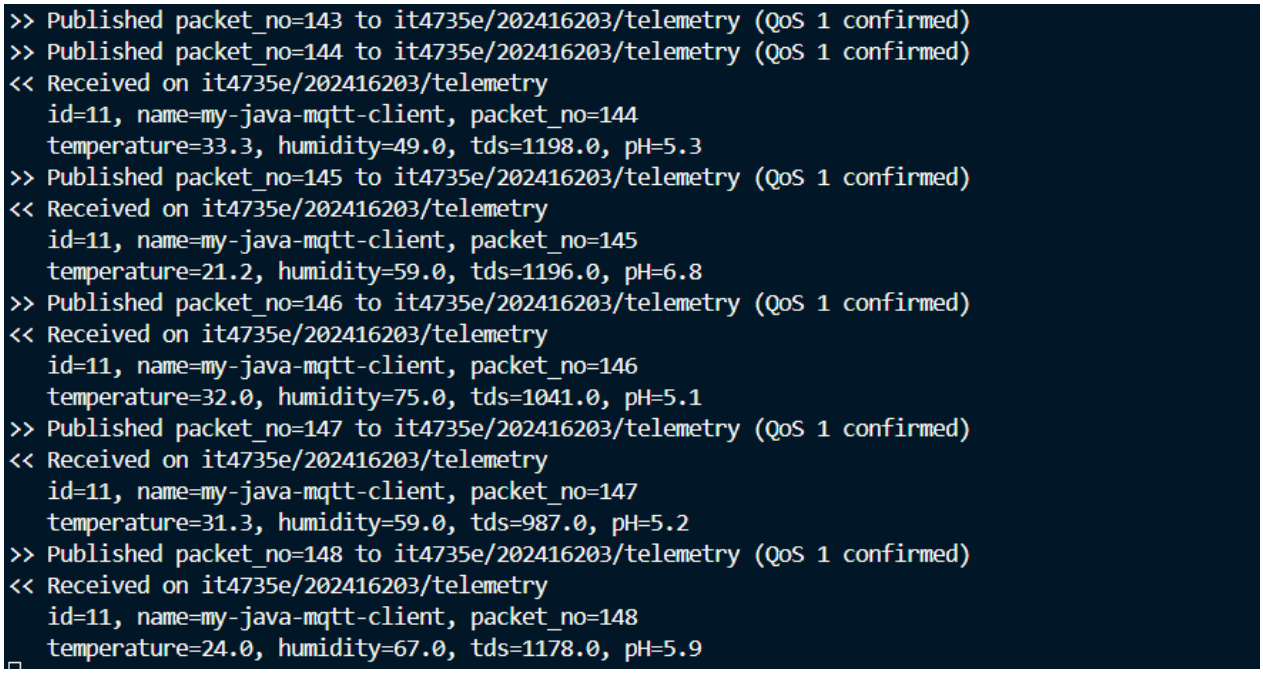
Hình 1 Java kết nối và đăng ký topic

Terminal hiển thị Client ID riêng, kết nối thành công tới HiveMQ và đăng ký it4735e/202416203/# với QoS 1. Java gửi trạng thái online có retained và bắt đầu publish bản tin telemetry đầu tiên với packet_no=126.



Hình 2 Java gửi và nhận telemetry định kỳ

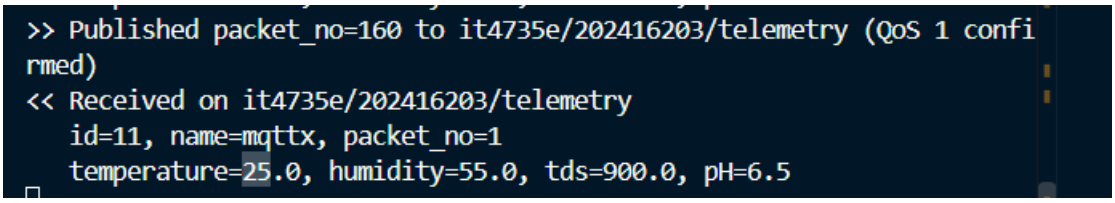
Các bản tin trên it4735e/202416203/telemetry có packet_no tăng từ 143 đến 148. Dòng QoS 1 confirmed cho biết broker đã xác nhận bản tin Java gửi. Do đăng ký topic có ký tự #, Java cũng nhận lại telemetry của chính mình và hiển thị đủ id, name, packet_no, temperature, humidity, tds và pH.

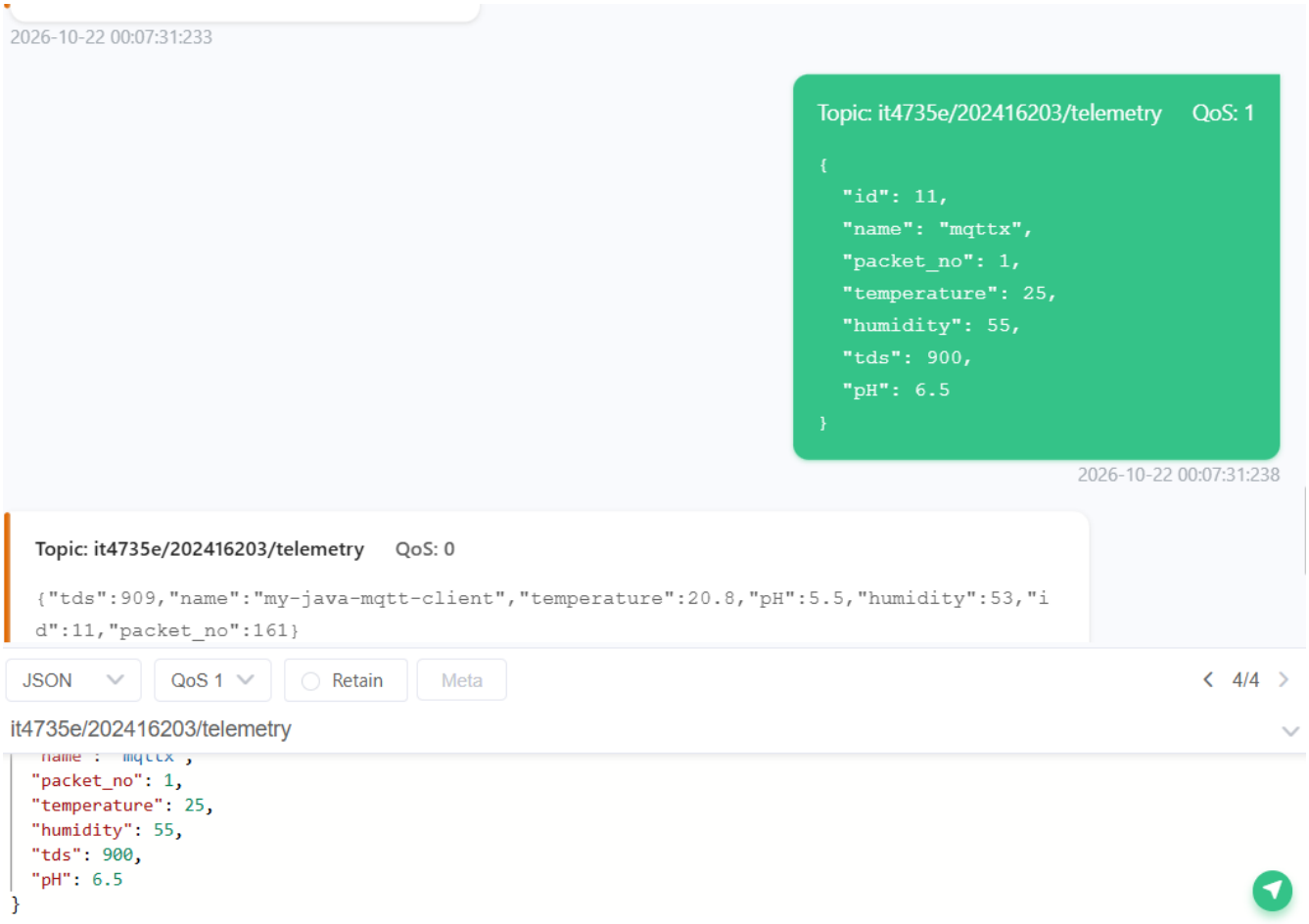


Hình 3 Java nhận và phân tích JSON gửi từ MQTTX

MQTTX publish bản tin JSON lên it4735e/202416203/telemetry với QoS 1. Payload kiểm thử gồm id=11, name=mqttx, packet_no=1, temperature=25, humidity=55, tds=900 và pH=6.5. Thẻ Published màu xanh trong MQTTX hiển thị bản tin đã gửi.

Terminal Java hiển thị Received trên cùng topic và đọc đủ bảy trường với các giá trị trùng khớp payload, trong đó name=mqttx và packet_no=1 phân biệt bản tin kiểm thử với telemetry do Java tự gửi. Kết quả xác nhận chương trình nhận bản tin từ client MQTTX qua HiveMQ và phân tích JSON để hiển thị dữ liệu.

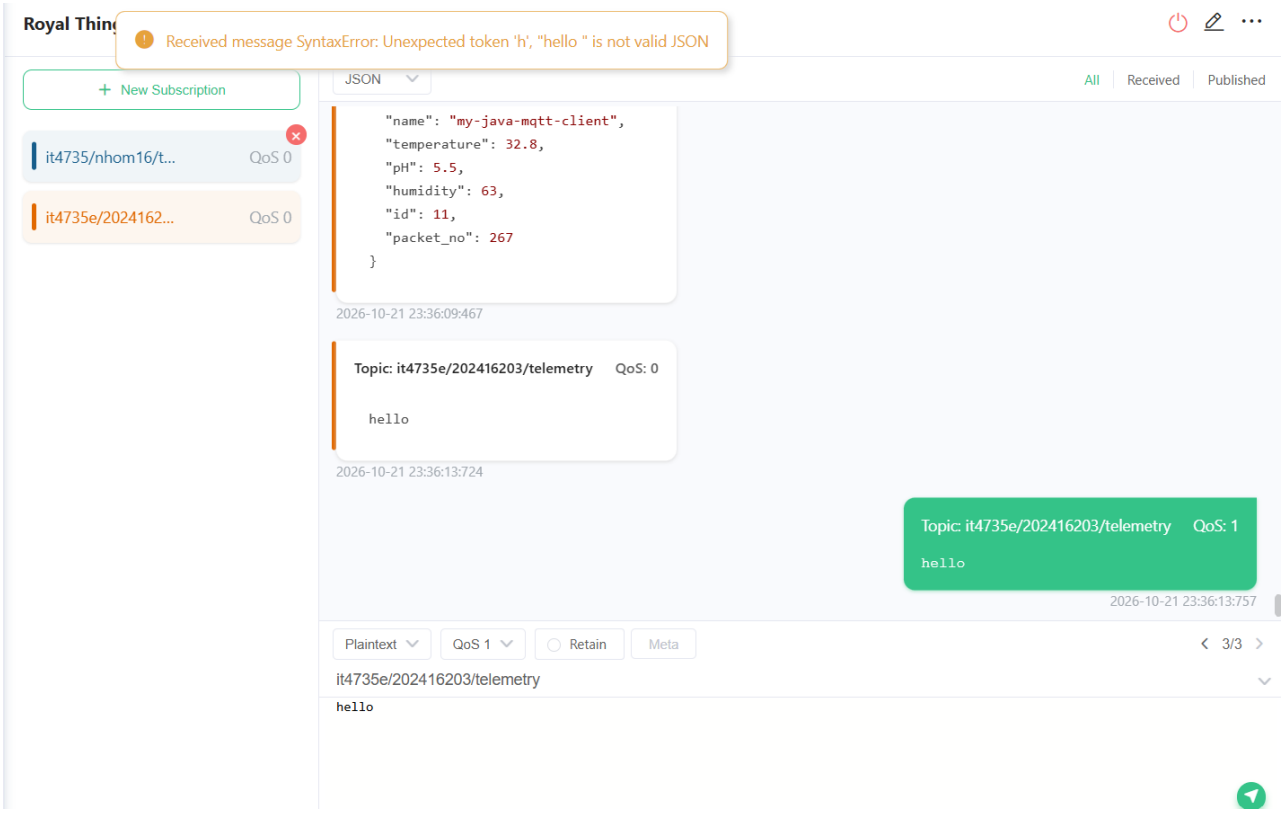




Hình 4 Xử lý bản tin sai JSON và tiếp tục hoạt động

MQTTX publish chuỗi hello dưới dạng Plaintext vào it4735e/202416203/telemetry với QoS 1. Cảnh báo Received message SyntaxError trên MQTTX xuất hiện do công cụ đang hiển thị bản tin nhận ở chế độ JSON.

Trong terminal Java, payload này gây thông báo JSON error kèm continuing. Ngay sau đó, các bản tin packet_no=268 và 269 tiếp tục được gửi, xác nhận QoS 1 và nhận lại bình thường. Lỗi JSON được xử lý mà không phải khởi động lại chương trình.

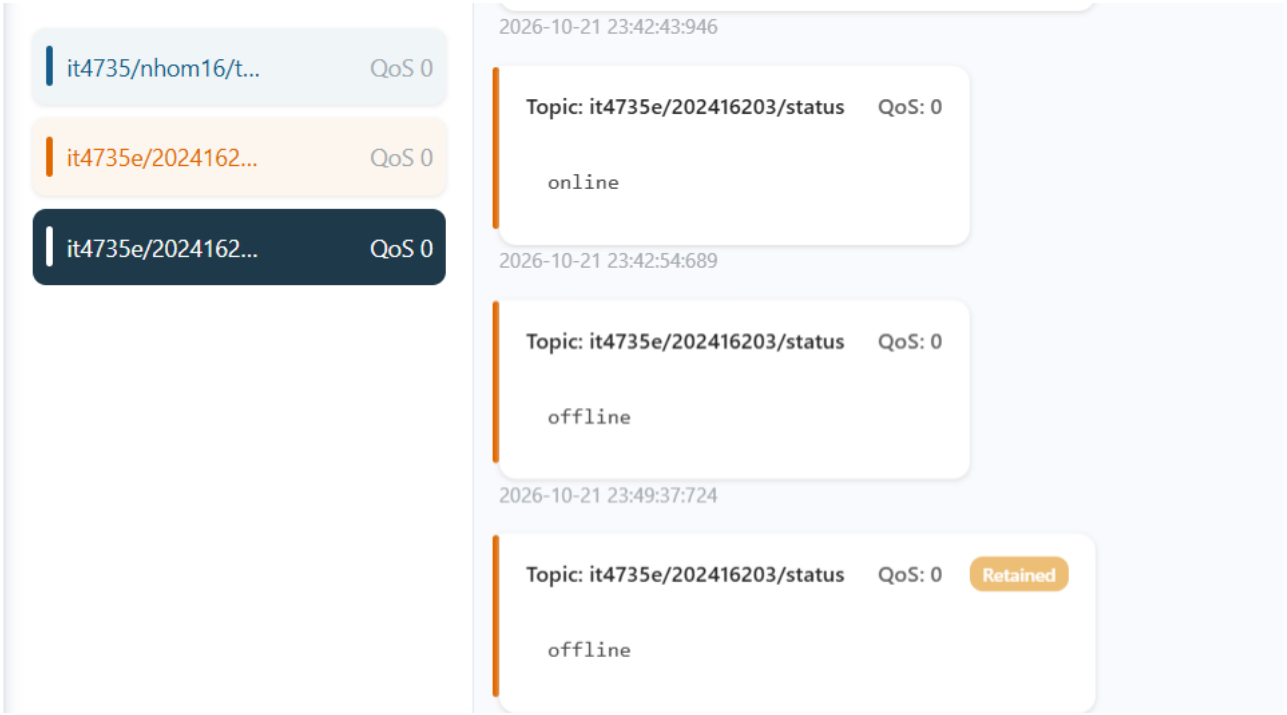



```
>> Published packet_no=267 to it4735e/202416203/telemetry (QoS 1 confirmed)
<< Received on it4735e/202416203/telemetry
   id=11, name=my-java-mqtt-client, packet_no=267
   temperature=32.8, humidity=63.0, tds=1023.0, pH=5.5
JSON error on it4735e/202416203/telemetry: A JSONObject text must begin with '{' at 1 [character 2 line 1] (con
tinuing)
>> Published packet_no=268 to it4735e/202416203/telemetry (QoS 1 confirmed)
<< Received on it4735e/202416203/telemetry
   id=11, name=my-java-mqtt-client, packet_no=268
   temperature=27.1, humidity=74.0, tds=843.0, pH=5.9
>> Published packet_no=269 to it4735e/202416203/telemetry (QoS 1 confirmed)
<< Received on it4735e/202416203/telemetry
   id=11, name=my-java-mqtt-client, packet_no=269
   temperature=25.2, humidity=58.0, tds=1193.0, pH=5.8
```

Hình 5 Last Will và trạng thái retained

MQTTX theo dõi it4735e/202416203/status khi Java đang online. Sau khi tiến trình Java bị kết thúc bất thường bằng Task Manager, broker phát hiện mất kết nối và gửi Last Will có payload offline lên topic trạng thái.

Khi đăng ký lại topic status, MQTTX nhận ngay offline với nhãn Retained. Broker đã lưu trạng thái cuối cùng và gửi lại cho subscription mới. Tin online được Java chủ động gửi lúc kết nối; Last Will được cấu hình là offline.



Hình 6 Tự kết nối lại và đăng ký lại sau khi mất mạng

Khi mạng bị ngắt tạm thời, Java báo lỗi xác nhận gửi, Connection lost và tạm dừng publish. Sau khi mạng được khôi phục, chương trình tự kết nối lại tới HiveMQ, hiển thị Reconnected và đăng ký lại it4735e/202416203/# với QoS 1.

Java gửi lại trạng thái online có retained và tiếp tục gửi, nhận telemetry với packet_no=138, 139 và 140. Các bước phục hồi được thực hiện trong cùng phiên chạy, không cần khởi động lại chương trình.

```
>> Published packet_no=134 to it4735e/202416203/telemetry (QoS 1 confirmed)
<< Received on it4735e/202416203/telemetry
   id=11, name=my-java-mqtt-client, packet_no=135
   temperature=32.1, humidity=58.0, tds=1125.0, pH=6.9
>> Published packet_no=135 to it4735e/202416203/telemetry (QoS 1 confirmed)
Publish not confirmed: Timed out waiting for a response from the server. Program continues.
Publish not confirmed: Connection lost. Program continues.
Connection lost: Connection lost. Automatic reconnect is enabled; publishing is paused.
Reconnected to tcp://broker.hivemq.com:1883
Subscribed: it4735e/202416203/# (QoS 1)
<< Status on it4735e/202416203/status: offline
>> Published retained online to it4735e/202416203/status
<< Status on it4735e/202416203/status: online
>> Published packet_no=138 to it4735e/202416203/telemetry (QoS 1 confirmed)
<< Received on it4735e/202416203/telemetry
   id=11, name=my-java-mqtt-client, packet_no=138
   temperature=30.5, humidity=69.0, tds=1147.0, pH=6.8
>> Published packet_no=139 to it4735e/202416203/telemetry (QoS 1 confirmed)
<< Received on it4735e/202416203/telemetry
   id=11, name=my-java-mqtt-client, packet_no=139
   temperature=25.1, humidity=40.0, tds=808.0, pH=5.7
<< Received on it4735e/202416203/telemetry
   id=11, name=my-java-mqtt-client, packet_no=140
   temperature=23.9, humidity=59.0, tds=875.0, pH=6.4
>> Published packet_no=140 to it4735e/202416203/telemetry (QoS 1 confirmed)
```

Nhận xét sau khi thực hành: .....